Federated Elections with Encrypted Local Tallies and Verifiable On-Chain Aggregation

Vincenzo Imperati[1], Lorenzo Camilli[1], and Raffaele Ruggeri[1]

Sapienza University of Rome, Italy
imperati@di.uniroma1.it, camilli@di.uniroma1.it, ruggeri@di.uniroma1.it

Abstract. We study federated elections in which voting remains secret and counted locally, but the global result must be derived from all local tallies without trusting a central coordinator and without exposing local outcomes. Publishing local tallies in the clear fixes inclusion but leaks sensitive local political information; commit-and-reveal only delays that leakage and still leaves a final-stage abort surface. We present a blockchain-assisted protocol in which each authority submits an *encrypted* local tally vector together with a Groth16 proof that the hidden tally is structurally valid: ciphertexts are well formed, entries are non-negative, and the vector sums to the public local electorate size. Authorities also sign the submission transcript, so accepted ciphertexts are bound to the current election state. The contract verifies the local-tally proof, records one signed submission per authority, and maintains the homomorphic aggregate on-chain; separate proofs check trustee setup and threshold opening. Only the global tally is recovered at the end. We provide a reproducible artifact with a Solidity contract, three Circom circuits, and an end-to-end replay of accepted submissions, on-chain aggregation, and final tally recovery without ever revealing local tallies.

Keywords: Federated elections · Tally integrity · Zero-knowledge proofs · Homomorphic aggregation

1 Introduction

We study federated elections in which authorized local authorities conduct secret local voting and compute local tallies off-chain. Ballot secrecy is already enforced locally and local committees are assumed to count correctly. The weak point is the transition from local results to the global result: a central coordinator can omit unfavorable tallies, misreport the aggregate, or exploit early knowledge of partial results.

DeCifris Template – Contribution

Publishing local tallies on a blockchain fixes inclusion but exposes local outcomes, which can create retaliation, reputational pressure, or strategic bargaining, especially in small electorates. Commit-and-reveal only postpones that disclosure and still leaves a final-stage abort surface.

This motivates the central design choice of this paper: *the local tally should remain encrypted throughout the protocol*. Each authority publishes an encrypted tally vector and a zero-knowledge proof that the hidden tally is structurally valid and sums to its public electorate size. The contract accepts one such submission per authority, fixes the transcript on-chain, computes the homomorphic aggregate, and only the global final tally is decrypted.

Our design is closer to open-audit bulletin board systems such as Helios [1] and to multi-authority election protocols [3] than to a ballot-level remote voting stack, but adds three elements for private local tallies: validity proofs for encrypted local results, an on-chain aggregate updated from accepted submissions, and a proof-checked threshold opening transcript that reveals only the final aggregate.

We use curve-based additive ElGamal rather than Paillier [8] because the artifact is proof-heavy and benefits from aligning encryption with Circom/Groth16 arithmetic. We remain at the local-tally level because the problem addressed here is final aggregation from trusted local counts, not ballot-level remote voting.

The paper specifies a protocol with encrypted local tallies, signed submission binding, proof-based acceptance, on-chain homomorphic aggregation, and threshold final opening. It also provides a reproducible artifact that exercises trustee setup, encrypted submission, on-chain aggregation, partial decryption validation, and final tally recovery.

2 System Model and Attacks

The protocol has four roles. The *owner* initializes the election and the authority registry. Each *local authority* computes its local tally off-chain and submits one encrypted tally vector. The *smart contract* acts as the public bulletin board and proof gate. A small set of *trustees* performs threshold decryption of the final aggregate.

Our assumptions are deliberately narrow. Ballots remain secret inside each authority, local counting is outside the protocol boundary, electorate sizes are known to the contract, and the final result must be derived from the full set of accepted local tallies rather than from a coordinator's private view. We add one privacy requirement that cleartext bulletin boards do not satisfy: local tallies should remain confidential even after the election.

These assumptions yield five attack classes: *omission and replacement* by a coordinator; *strategic observation* if local tallies become visible too early; *local retaliation* if local tallies remain public after the election; *malformed encrypted submissions* that hide negative, oversized, or wrong-sum tallies; and *opening inconsistencies* in

which either the trustee registry is not consistent with the election public key or the decryption transcript contains invalid shares.

The protocol addresses omission and replacement through encrypted submission and transcript inclusion, and it addresses strategic observation and local retaliation by ensuring that local tallies are never revealed and that only the final aggregate is opened. Malformed encrypted submissions are excluded through zero-knowledge proofs, while opening inconsistencies are constrained through signed submission binding and proof-checked setup and opening transcripts for trustees.

The remaining limitations are explicit. A proof of *well-formedness* is not a proof that the encrypted tally is the *true* local tally, so a fully corrupted local committee remains outside scope. The artifact also uses a dealer-generated threshold key rather than distributed key generation, and final message decoding after threshold opening is performed off-chain through bounded discrete-log recovery.

3 Protocol Overview

Setup. The contract records the authorized local authorities, the electorate size of each authority, an election identifier, a trustee set with public share metadata, a decryption threshold, and a public key for additively homomorphic ElGamal-style encryption [4] over a curve group. Before the election starts, the owner also provides a Groth16 proof that the registered trustee public shares are consistent with the threshold setup associated with that election public key [9,5].

Encrypted local tally. Authority A_i computes off-chain counts $(t_{i,1}, \ldots, t_{i,m})$. Instead of publishing them, it encrypts each component and obtains a ciphertext vector $C_i = (\mathsf{Enc}(t_{i,1}), \ldots, \mathsf{Enc}(t_{i,m}))$. Because the encryption is additively homomorphic, component-wise aggregation of ciphertexts corresponds to component-wise addition of the hidden tallies.

Zero-knowledge validity proof. Together with C_i, the authority submits a Groth16 proof π_i [7] for the statement that there exist plaintext counts and encryption randomness such that the published ciphertexts encrypt those counts, each count is in range, and the sum of the counts equals the public electorate size of A_i. The contract complements that proof with an authority signature on the submission transcript. The signature binds the current election identifier, authority address, electorate size, and ciphertext coordinates to the sender, so the accepted submission is anchored both cryptographically and institutionally.

On-chain acceptance. The contract accepts exactly one encrypted tally per authorized authority. A submission is accepted only if it arrives before the deadline, has not already been submitted by that authority, matches the expected public inputs, and passes on-chain Groth16 verification. Once accepted, the ciphertext vector is

part of the immutable transcript and is immediately folded into an on-chain aggregate ciphertext maintained by BabyJub point addition.

Deterministic aggregation. No privileged actor posts the aggregate on behalf of everyone else. Instead, the contract maintains the component-wise homomorphic sum of accepted ciphertexts as submission state evolves. Any observer can reconstruct the same aggregate from the accepted authority transcript and compare it with the contract-maintained aggregate.

Threshold decryption. Only the aggregate ciphertext is opened. The design follows threshold decryption for voting protocols [6]. For each candidate component, trustees publish a share point together with a Groth16 proof that the share is consistent with both the trustee's public share and the corresponding contract-maintained ciphertext. The contract accepts only verified shares. Once a threshold of valid shares is available, any observer can reconstruct the final tally and determine the winner. In the artifact, the last message decoding step is performed off-chain through bounded discrete-log recovery, while the acceptance of decryption shares is already enforced on-chain.

Failure handling. If the submission deadline expires before all authorities have posted an accepted encrypted tally, the election is cancelled instead of producing a partial result. This preserves the invariant that the final result must come from the complete accepted set of local tallies.

4 Implementation and Artifact

The companion artifact is available in the public repository[1]. The repository contains a Solidity contract that registers authorities, verifies Groth16 proofs on-chain, fixes the encrypted-tally transcript, maintains the aggregate ciphertext, and anchors both the trustee setup and the final opening transcript. It also contains three Circom circuits [2]: one for encrypted local tally validity, one for trustee-registry consistency with the election public key, and one for partial decryption share validity.

The artifact's reviewer entry point is `yarn validate`. That command rebuilds the circuit artifacts if needed, runs the contract test suite, and executes an end-to-end replay with three authorities, a two-candidate election, a 2-of-3 trustee committee, and a dealer-generated threshold key. The replay generates real proofs, submits signed encrypted local tallies, reconstructs the aggregate from the stored transcript, checks it against the on-chain aggregate, and recovers the final tally.

The local tallies are never published in clear. The public transcript contains only accepted ciphertexts, proof and signature checks, trustee setup, verified decryption shares, and the final aggregate after threshold opening.

[1] `https://github.com/VincenzoImp/encrypted-local-tally-integrity`.

5 Discussion and Conclusion

Origin. The idea comes from the Cryptography track of the 2024 De Cifris/XRPL Commons hackathon, whose federated-election scenario had a specific trust gap: local committees already preserve ballot secrecy and count local votes, while the unresolved risk is the central actor who collects tallies and announces the final result. This led us to focus on federation-wide aggregation integrity rather than replacing local voting.

Approach. The protocol separates local tallying from public aggregation. Local authorities publish signed encrypted tally vectors with validity proofs; the contract accepts one proof-checked submission per authority, maintains the homomorphic aggregate, and verifies the threshold-opening transcript before the global result is recovered.

Product path. The artifact suggests a practical route for institutional governance systems, provided deployment engineering is added, including operational key management, audited contracts and circuits, authority onboarding, monitoring, and interfaces for administrators. The present contribution remains a research prototype, but it demonstrates a reproducible workflow for private local tallies and verifiable global aggregation.

References

1. Adida, B.: Helios: Web-based open-audit voting. In: van Oorschot, P.C. (ed.) USENIX Security 2008. pp. 335–348. USENIX Association (Jul / Aug 2008), http://www.usenix.org/events/sec08/tech/full_papers/adida/adida.pdf
2. Circom developers: Circom 2 documentation. https://docs.circom.io (2022)
3. Cramer, R., Gennaro, R., Schoenmakers, B.: A secure and optimally efficient multi-authority election scheme. In: Fumy, W. (ed.) EUROCRYPT'97. LNCS, vol. 1233, pp. 103–118. Springer, Berlin, Heidelberg (May 1997). https://doi.org/10.1007/3-540-69053-0_9
4. ElGamal, T.: A public key cryptosystem and a signature scheme based on discrete logarithms. IEEE Transactions on Information Theory **31**(4), 469–472 (1985). https://doi.org/10.1109/TIT.1985.1057074
5. Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: 28th FOCS. pp. 427–437. IEEE Computer Society Press (Oct 1987). https://doi.org/10.1109/SFCS.1987.4
6. Fouque, P.A., Poupard, G., Stern, J.: Sharing decryption in the context of voting or lotteries. In: Frankel, Y. (ed.) FC 2000. LNCS, vol. 1962, pp. 90–104. Springer, Berlin, Heidelberg (Feb 2001). https://doi.org/10.1007/3-540-45472-1_7
7. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part II. LNCS, vol. 9666, pp. 305–326. Springer, Berlin, Heidelberg (May 2016). https://doi.org/10.1007/978-3-662-49896-5_11

8. Paillier, P.: Public-key cryptosystems based on composite degree residuosity classes. In: Stern, J. (ed.) EUROCRYPT'99. LNCS, vol. 1592, pp. 223–238. Springer, Berlin, Heidelberg (May 1999). https://doi.org/10.1007/3-540-48910-X_16
9. Shamir, A.: How to share a secret. Communications of the Association for Computing Machinery **22**(11), 612–613 (Nov 1979). https://doi.org/10.1145/359168.359176